

M2 Command-Line Interface Result

October 9, 2026

Contents

1. Summary
2. Goal and completion criteria
3. Work done
4. Results
5. Discussion
6. Next steps

References

Purpose. This document reports the result of the command-line interface built according to the M2 design [D1]. Because M2 is the last milestone, it also judges the overall success criteria of the goal from the proposal, and asks for approval to close M2 and this goal.

1 Summary

A `main()` was added to `convert.py`, so `python convert.py 100 km mi` prints `62.13711922 mi`. **All 5 M2 completion criteria and all 4 proposal success criteria were met.**

- **Tests.** 10 CLI tests that launch a real process and 127 M1 tests, 137 in total, passed. They also passed when run from a different folder.
- **Problem found by running it directly.** A non-numeric value produced “invalid _finite value”, which leaked an internal function name. The test checked only for the word “invalid” and missed this. The message was changed to “not a number: 'abc'”, and the test now checks the full wording.
- **Remaining limitation.** On Python 3.11, `-1e3 m km` still gives the misleading error “not a number: 'm'”. As designed, only a `--` note was added to the help.

2 Goal and completion criteria

Table 1 lists the completion criteria from Table 5 of the M2 design and their verdicts [D1]. Table 2 lists the overall success criteria of the goal from Table 4 of the proposal and their verdicts [D3].

Table 1. M2 completion criteria and verdicts. [D1, D4]

Criterion	Target	Result	Verdict
Normal output	3 / 3 (100 km mi, -40 C F, -- -1e3 m km)	3 / 3	met
Errors	6 / 6, exit 2, empty standard output	6 / 6	met
Help	1 / 1, unit list and -- note	1 / 1	met
Regression	M1 tests 7 / 7	7 / 7	met
Dependencies	argparse, math only	argparse, math	met

Table 2. Proposal success criteria (whole goal) and verdicts. [D3, D4, D5]

Criterion	Target	Result	Verdict
Accuracy	relative error $\leq 1e-12$	max 1.4e-15 (M1)	met
Round-trip consistency	relative error $\leq 1e-12$	109 pairs, max 1.4e-16 (M1)	met
Error handling	4 / 4 give a message and exit code $\neq 0$	4 kinds + 2 kinds (nan, inf), all exit 2 with a reason	met
Dependencies	standard library only	argparse, math	met

3 Work done

The order from the design was followed (tests first $\rightarrow$ `main()` $\rightarrow$ full run and direct check). As in M1, the verdict vocabulary is adopted / option / rejected / deferred. On the first test run, 10 CLI subtests failed. This was the intended failure, because `main()` did not exist yet [D4].

3.1 Input and output (convert.py)

① Value conversion that accepts only finite numbers (M2.1) — adopted

Reason. `float()` accepts "nan" and "inf" as numbers, and they flow straight through to the conversion (design Table 2) [D1].

Method. The function `_finite()` was given as the type of `argparse`. It converts with `float()` and raises `ArgumentTypeError` if `math.isfinite()` is false [R1, R2].

Result. Both `nan km mi` and `inf km mi` ended with an error of the form “value must be a finite number, got 'nan'” and exit code 2.

② 10-significant-digit output and unified errors (M2.2) — adopted

Reason. The aim was to hide floating-point residue (211.99999999999994) from the user, and to make input parsing errors and conversion errors end in the same form [D1].

Method. The result is printed with `f"{result:.10g} {dst}"` [R3]. A `ValueError` from `convert()` is passed to `parser.error()` [R1]. A unit list built from the M1 `KINDS` dictionary and a -- note were added at the end of the help.

Result. `100 C F → 212 F`. All 6 error kinds ended with one usage line, one `convert.py: error: ... line`, and exit code 2 (Table 3).

3.2 Fixes from the direct check

③ Error message exposing an internal function name (M2.3) — adopted

Reason. After all tests passed, a direct run in the shell showed that `abc km mi` gave “argument value: invalid _finite value: 'abc'”. When a `type` function raises `ValueError`, `argparse` puts the function name into the message. The test checked only for the word “invalid”, so it missed this.

Method. Inside `_finite()`, the `ValueError` from `float()` is caught and turned into `ArgumentTypeError("not a number: 'abc'")`. The expected wording in the test was also changed from “invalid” to “not a number: 'abc'”, so the old message would fail.

Result. The message became “argument value: not a number: 'abc'”, and all 10 test methods passed again. A description (“number to convert”) was also added to the `value` argument in the help.

④ Better -1e3 error message (M2.4) — deferred

Reason. On Python 3.11, `-1e3 m km` treats `-1e3` as an option and reads `m` as the value [R1]. So it currently prints “not a number: 'm'”, which makes it hard to tell what went wrong.

Method. As decided in the design, only the note “negative numbers in exponent form need '--' first” was added at the end of the help. Attaching this note to the error message itself (one extra line) is also possible, but it was not done because it is outside the design scope.

Result. `-- -1e3 m km` → `-1 km` works correctly. The message improvement is left as an option in Section 6. Python 3.14 and later do not have this problem [R1].

4 Results

Table 3 is a record of direct runs in the shell. Whether the output is easy to read was judged by eye. Figure 1 shows the change in code size compared with M1.

Table 3. Shell run record. Windows 11, Python 3.11.16, 2026-10-09. Only the last line of each error is shown. The usage line `usage: convert.py [-h] value src dst` precedes it. [D4]

Input	Output	Exit
100 km mi	62.13711922 mi	0
100 C F	212 F	0
72 in cm	182.88 cm	0
1 lb oz	16 oz	0
1e-7 km mm	0.1 mm	0
-- -1e3 m km	-1 km	0
abc km mi	convert.py: error: argument value: not a number: 'abc'	2
nan km mi	... value must be a finite number, got 'nan'	2
1 km xx	... unknown unit 'xx' (known: mm, cm, m, km, ..., c, f, k)	2
1 KM KG	... cannot convert length to mass	2
-300 c f	... -300 c is below absolute zero	2
-1e3 m km	... not a number: 'm' (Section 3.2 ④)	2

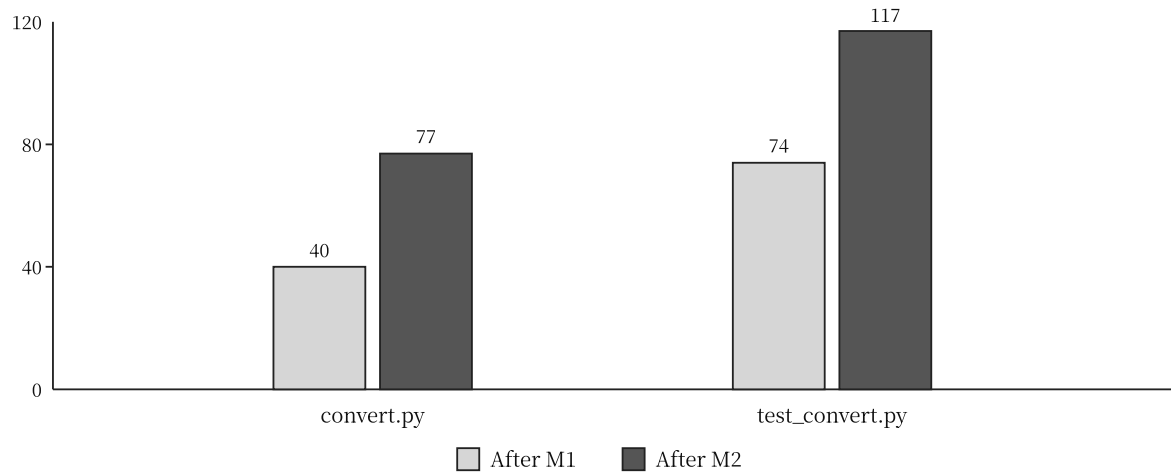


Figure 1. Lines per file (after M1 and after M2). `convert.py` grew by 37 lines, and the conversion arithmetic in it did not change. The growth is input parsing, output, and help. The test file grew by 43 lines of CLI tests.

Table 4. Measurements. [D4]

Item	Measurement	Value
Tests	methods / checks (M1 127 + CLI 10)	10 / 137
Test run time	<code>python -m unittest</code> , including 10 process launches	0.65 s
Single CLI run time	median / max of 20 runs	61.3 / 65.5 ms
Tests from another folder	<code>unittest discover -s <repository></code>	passed

5 Discussion

Passing tests and direct runs check different things. The message problem in Section 3.2 ③ arose because the test checked only for the word “invalid”. It surfaced only after one run in the shell. After the fix, all 6 error tests check the full reason wording.

The output format hides M1's floating-point residue. $-40\text{ }^{\circ}\text{C} \rightarrow\text{ }^{\circ}\text{F}$, which left a $1.4\text{e-}15$ error in M1, appears on screen as `-40` [D2]. The arithmetic did not change; only the displayed digits were limited to 10.

The 61 ms single CLI run time appears to be mostly Python startup time (not verified). A single conversion measured in M1 took less than $1\text{ }\mu\text{s}$.

The evidence also has limits. The tests were run only on Python 3.11.16 on Windows 11. Behavior on other operating systems and on Python 3.14 and later, and the effect of Windows console encoding, were not checked (not verified). Since the output is ASCII only, no effect is expected.

6 Next steps

All milestones of this goal are finished. Table 5 lists the remaining options. The code and documents are not committed yet [D4].

Table 5. Next steps.

Task	Content	Needed
Commit	Commit <code>convert.py</code> , <code>test_convert.py</code> , <code>docs/reports/</code> (current branch is master)	Developer request
-1e3 message (option)	Append a one-line -- note to the “not a number” error	Developer decision
Temperature differences, compound units	Extensions left out of scope in the proposal	Start as a new goal

References

- [R1] Python Software Foundation. `argparse` — Parser for command-line options, arguments and subcommands. <https://docs.python.org/3/library/argparse.html>
- [R2] Python Software Foundation. `math.isfinite`.
<https://docs.python.org/3/library/math.html#math.isfinite>
- [R3] Python Software Foundation. Format Specification Mini-Language.
<https://docs.python.org/3/library/string.html#format-specification-mini-language>
- [D1] M2 Command-Line Interface Design (`docs/reports/unit-converter-cli/M2-design.pdf`), approved 2026-10-09.
- [D2] M1 Conversion Core Result (`docs/reports/unit-converter-cli/M1-result.pdf`), approved 2026-10-09.
- [D3] Unit Conversion CLI Proposal (`docs/reports/unit-converter-cli/00-proposal.pdf`), approved 2026-10-09.
- [D4] `convert.py`, `test_convert.py`, and run and measurement records, Python 3.11.16, 2026-10-09 (uncommitted).
- [D5] Error measurements in the M1 result (Table 2 of D2).